Assignment: Custom Distributed Task Offloading & Remote GPU Rendering System

---

Course Code & Name: CSC-334: Parallel and Distributed Computing

Assignment Title: Custom Distributed Task Offloading & Remote GPU Rendering System

Total Marks: 100

Submission Deadline & Guidelines: Public GitHub Repository containing full source code, a comprehensive README.md, execution screenshots, and demonstration GIFs/videos.

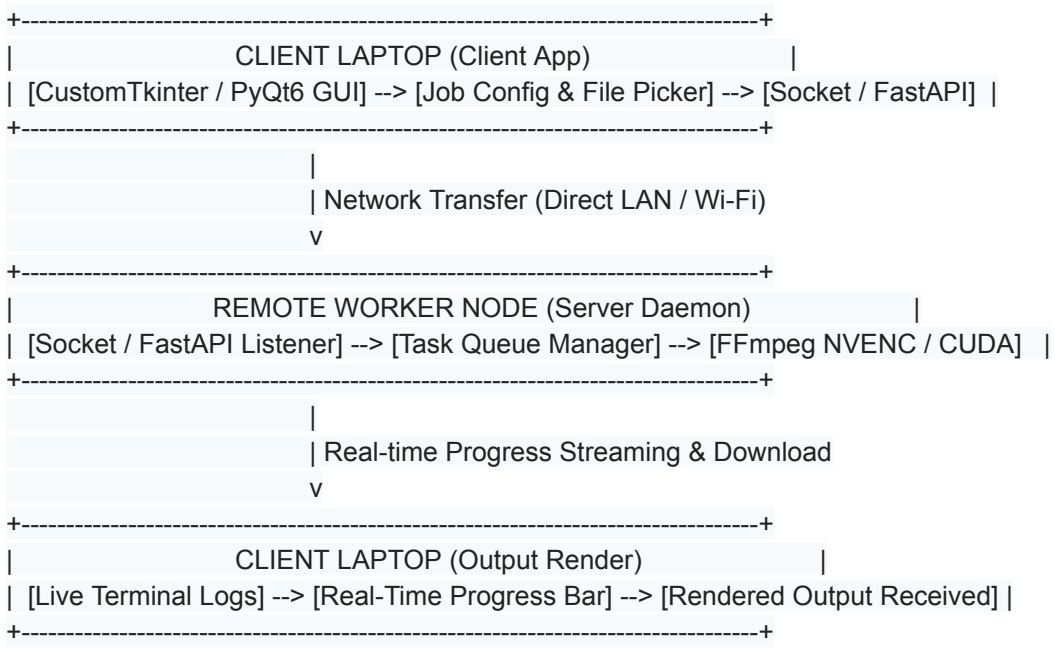
1. Problem Statement & Motivation

In modern computer science and multimedia workloads, students and developers frequently encounter hardware bottlenecks. Consider a real-world scenario where a developer is using a resource-constrained client laptop equipped only with an integrated GPU or a low-end discrete graphics card (e.g., 500MB VRAM). When tasked with heavy computational operations—such as high-resolution video transcoding, complex rendering, or fine-tuning deep learning models—the client machine suffers from extreme latency, thermal throttling, or outright out-of-memory (OOM) failures.

To overcome this hardware limitation without investing in expensive local hardware upgrades, engineers utilize **Distributed Task Offloading**. By establishing a high-speed local network connection (such as a Direct CAT6 Ethernet Cable or high-throughput Wi-Fi) between the resource-constrained client laptop and a remote worker node (e.g., a workstation or desktop equipped with a 4GB+ Dedicated NVIDIA GPU), heavy workloads can be serialized, transmitted, executed concurrently via hardware-accelerated engines, and streamed back seamlessly.

2. System Architecture & Workflow

The system follows a robust Client-Server architectural pattern. The client manages the graphical interface, job configuration, and file transmission, while the remote worker node acts as a headless daemon executing compute-heavy tasks using GPU acceleration.



3. Detailed Task Breakdown (100 Marks Total)

Task 1: Networking & Handshake Protocol Setup (20 Marks)

- Configure static IP addressing for direct peer-to-peer LAN connection or dedicated Wi-Fi subnet (e.g., Server: 192.168.1.1, Client: 192.168.1.2).
- Implement a robust socket connection handshake protocol, including an initial latency ping check and basic error detection to verify worker availability before job submission.

Task 2: Remote GPU Execution Engine Daemon (25 Marks)

- Develop a server-side Python background service or daemon that securely accepts render and compute configurations.
- Implement execution logic leveraging GPU hardware acceleration, specifically targeting FFmpeg with NVENC (-c:v h264_nvenc) for video transcoding or PyTorch CUDA engines for tensor operations.

Task 3: Client Desktop GUI Application (25 Marks)

- Design an intuitive desktop user interface utilizing modern frameworks such as **CustomTkinter** or **PyQt6**.
- Include core components: file picker for input assets, render quality settings (Resolution,

Bitrate, Preset), server IP configuration fields, and an integrated real-time log terminal.

Task 4: Real-Time Progress Tracking & Network Robustness (15 Marks)

- Implement asynchronous progress percentage streaming from the remote worker node back to the client application.
- Build resilience features to handle network timeouts, incomplete file transfer validation (checksum checks), and unexpected socket disconnects gracefully.

Task 5: Performance Benchmarking & Analysis Report (15 Marks)

- Conduct a thorough performance evaluation comparing local client rendering time against remote GPU offloading time across different file sizes and resolutions.
- Calculate the overall speedup factor, network transfer overhead, and resource utilization metrics in a formal technical report.

4. Marking Scheme & Evaluation Rubric

Component	Core Evaluation Criteria	Marks Allocated
Task 1: Networking & Handshake	Static IP configuration, connection handshake protocol, latency checking, and error handling.	20 Marks
Task 2: Remote GPU Daemon	Server background service, configuration parsing, and hardware-accelerated execution (NVENC/CUDA).	25 Marks
Task 3: Client GUI Application	Modern UI design (CustomTkinter/PyQt6), parameter controls, file picker, and live terminal logging.	25 Marks

Component	Core Evaluation Criteria	Marks Allocated
Task 4: Robustness & Progress	Asynchronous progress streaming, timeout management, file integrity validation, and graceful recovery.	15 Marks
Task 5: Benchmarking & Report	Comparative speedup analysis, network overhead evaluation, and technical report clarity.	15 Marks
Total	**Comprehensive System Implementation**	**100 Marks**

5. Submission Instructions & Repository Checklist

- Push the complete source code to a public GitHub repository.
- Structure the repository clearly with separate directories for client/ and server/.
- Include a thorough README.md featuring:
  1. Step-by-step setup instructions for both client and server nodes.
  2. Network configuration guide (static IP setup over Ethernet/Wi-Fi).
  3. Execution guide explaining how to start the daemon and launch the GUI.
  4. Demo screenshots and GIFs illustrating live progress streaming and output generation.
- Submit the official public GitHub repository link to the course portal before the deadline.